

## Quiz 2 Problem Set

Quiz 2 will be based on these parts of the textbook:

- Chapter 5: 5.1, 5.2, 5.3
- Chapter 7: 7.1, 7.2, 7.3

The questions on the quiz may refer to definitions, theorems, and examples from the textbook. The questions could be of any type, and will be variations of questions below, or questions that are similar.

**Important:** *Treat these problem-sets as non-AI activities!* Turn off all AI support and try to figure them out yourself **by hand**. Having AI or another student do this for you will not help you learn. You must do the learning yourself!

### Question 1

Write a **stack** abstract data type (ADT) using the operations below. For each operation, give the return-type, the type of the parameters (if any), and an  $O$ -notation expression for the worst-case performance of the operation in terms of  $n$  (the number of items in the stack). Assume the items on the stack are all of the same type  $T$ , but don't make any assumption about what  $T$  is.

This is an ADT, so there are no implementation details: the operations have *no* implementation, and there are *no* variables.

Operations: *push*, *pop*, *peek* (get copy of top), *size*, *is\_empty*, *clear\_all*

### Question 2

Implement two different stack classes: one based on an array of `strings` (not a `vector<string>`), and one based on a linked list of strings (e.g. a singly-linked list).

### Question 3

Write a program that takes an array of size  $n$  as input, and uses a stack to reverse the order of the elements in the array.

### Question 4

Write a program that reads a string of brackets as input, and uses a stack to check if the brackets are matched. The bracket pairs are “()”, “{}”, and “[ ]”.

## Question 5

Write a **queue** abstract data type (ADT) using the operations below. For each operation, give the return-type, the type of the parameters (if any), and an  $O$ -notation expression for the worst-case performance of the operation in terms of  $n$  (the number of items in the queue). Assume the items in the queue are all of some type  $T$ , but don't make any assumption about what  $T$  is.

This is an ADT, so there are no implementation details: the operations have *no* implementation, and there are *no* variables.

Operations: *enqueue*, *dequeue*, *peek\_front*, *size*, *is\_empty*, *remove\_all*

## Question 6

Implement two different queue classes: one based on an array of strings (not a `vector<string>`), and one based on a linked list of strings (e.g. a doubly-linked list).

## Question 7

- a) Given a **stack**  $S$  with 0 or more items, write a fragment of code that prints the number of items in  $S$ . Assume  $S$  does not have a `size()` operation.  $S$  should end up in exactly the same state as before calling the code. Only use **stacks** in your answer: do **not** use arrays, vectors, etc.
- b) Given a queue  $Q$  with 0 or more items, write a fragment of code that prints the number of items in  $Q$ . Assume  $Q$  does not have a `size()` operation.  $Q$  should end up in exactly the same state as before calling the code. Only use **queues** in your answer: do **not** use arrays, vectors, etc.

## Question 8

Write an ADT for a **double-ended queue** (a **deque**). Include the operation return types and parameter types. Also, for each operation give an  $O$ -notation expression for the worst-case performance of that operation.

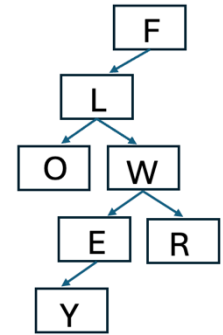
## Question 9

Give the formal definition of a tree, and define each of these terms: *internal node*, *external node*, *sibling*, *leaf (leaves)*, *ancestor*, *descendant*, *path*, *ordered tree*, *binary tree*.

## Question 10

a) Answer each question for the binary tree on the right:

- What is its height?
- In what order are the nodes printed using a **preorder** traversal?
- In what order are the nodes printed using an **inorder** traversal?
- In what order are the nodes printed using a **postorder** traversal?



b) For a binary tree with 0 or more nodes, give C++-like pseudocode for each of these operations:

- the **height** of the binary tree
- a **preorder** traversal of the binary tree (printing the nodes)
- an **inorder** traversal of the binary tree (printing the nodes)
- a **postorder** traversal of the binary tree (printing the nodes)

## Question 11

Prove propositions 7.10 and 7.11 from the textbook.